# CSS3 | Kaskade und Vererbung

# CSS3 | Kaskade und Vererbung

# CSS3 | Kaskade (Cascading)

Bei der Gestaltung eines Elements sammelt der Browser zunächst für jede CSS-Eigenschaft alle relevanten Anweisungen und schreibt sie auf einen metaphorischen Zettel. Wenn es für eine Eigenschaft **mehrere Anweisungen** gibt, durchläuft er auf der Suche nach einem eindeutigen Wert einen dreistufigen **Entscheidungsprozess namens Kaskade**:

1. **Zuerst** sortiert der Browser **nach** Wichtigkeit. Er schaut, ob es eine Anweisung mit **!important** gibt. Gibt es genau eine, nimmt er den Wert aus dieser Anweisung und beendet die Kaskade.
2. Gibt es keine oder mehrere Anweisungen mit !important, **prüft** er die **Spezifität**. Hat genau eine Anweisung mehr Punkte als alle anderen, ist die Kaskade beendet.
3. Haben mehrere Anweisungen dieselbe Punktzahl, geht es um die Reihenfolge im CSS, und da kann es kein Unentschieden geben.
   **Der Browser nimmt den Wert aus der zuletzt notierten Anweisung.**

```
.rot{
    color: red !important;
}
```

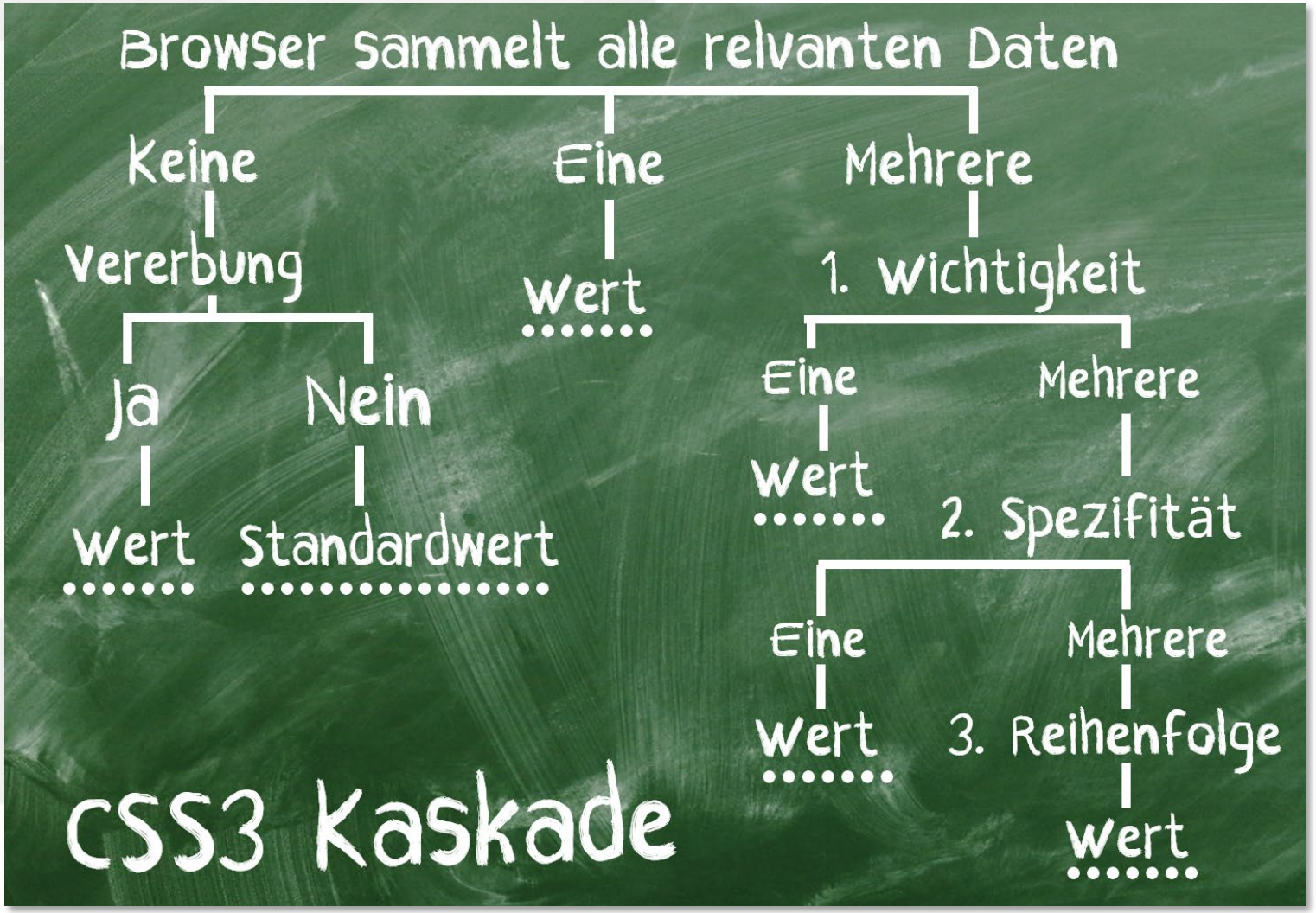
**Ziel dieser Kaskade ist es, genau eine Anweisung zu finden, die der Browser dann anwendet**

# CSS3 | Kaskade (Cascading)

Neben der Vererbung ist die **Kaskade** ein unverzichtbares Thema in CSS.

Schließlich steht bei CSS das C für Cascading (deutsch: Kaskade).

Wenn Sie das Prinzip der Kaskade verstanden haben, können Sie ein Stylesheet auf das andere aufbauen und sich dabei eine Menge Tipparbeit ersparen.



# CSS3 | Kaskade (Cascading)

**Mit Kaskade ist gemeint, dass ein Dokument nicht nur von einem Stylesheet, sondern aus einer Vielzahl von Stylesheets formatiert werden kann, die aus verschiedenen Quellen kommen können.**
Dadurch wird es möglich, dass ein Stylesheet – richtig eingesetzt – auf das andere aufbaut und Sie sich so eine Menge Arbeit ersparen. Nun ist es so, dass es durch die vielfältigen Möglichkeiten, die Stylesheetanweisungen einzubinden und miteinander zu kombinieren, zu Konflikten und Widersprüchen kommen kann.

**Ein solcher Konflikt entsteht, wenn bei mehreren Anweisungen dieselbe CSS-Eigenschaft mit unterschiedlichen Werten belegt wurde. Für solche Fälle gibt es ein Regelsystem, nach dem entschieden wird, welche der widersprüchlichen bzw. konkurrierenden Stilanweisungen auf ein Element letztendlich ausgeführt werden.**

```
<link href="css/bootstrap-4.3.1.css" rel="stylesheet">
<link href="css/layout.css" rel="stylesheet" type="text/css">
</head>
```

# CSS3 | Kaskade (Cascading)

**Die Herkunft eines Stylesheets**

Ein Stylesheet kann von drei verschiedenen Quellen kommen: **Webbrowser, Benutzer und Autor.**

- **Browserstylesheet**: Das **Standardstylesheet des Webbrowsers** wird verwendet, wenn keine CSS-Formatierung dem HTML-Dokument zugewiesen ist. **Jeder Webbrowser stellt hierfür eine Grundformatierung zur Verfügung**, bei der die Schriftgrößen für Überschriften, Unterstreichungen für Links, Einrückungen für Listen, Abstände bei Absätzen und noch vieles mehr mit Standardwerten belegt sind. Jeder Webbrowser liefert seine eigene Standardeinstellung mit, sodass es gewöhnlich zu leichten Differenzen kommt.
- **Benutzerstylesheet**: Einige Webbrowser bieten dem Benutzer direkt oder über Erweiterungen an, eigene Stylesheetdateien einzubinden. Wenn der Benutzer eigene Stylesheets (Benutzerstylesheets, *Userstyles*) einbindet, werden die entsprechenden Eigenschaften des Webbrowsers überschrieben.
- **Autorenstylesheet**: Das ist das von Ihnen angelegte und referenzierte (**verlinkte**) oder mit der @import-Regel eingebundene Stylesheet, mit dem gewöhnlich verschiedene CSS-Eigenschaften des Standardstylesheets (des Browserstylesheets) (und gegebenenfalls des Benutzerstylesheets) überschrieben bzw. kombiniert werden.

# CSS3 | Kaskade (Cascading)

**Von diesen drei Quellen hat das Browserstylesheet die niedrigste Priorität**. Wird ein Benutzerstylesheet verwendet – das eine mittlere Priorität hat –, wird das Browserstylesheet überschrieben.
**Das Autorenstylesheet hingegen hat die höchste Priorität und überschreibt sowohl das Browser- als auch das Benutzerstylesheet.** Vereinfacht ausgedrückt, ergeben sich durch diese drei Quellen drei Möglichkeiten, CSS-Eigenschaften auf ein HTML-Element anzuwenden:

1. Es wird **keine CSS-Eigenschaft** für ein HTML-Element deklariert, dann wird überprüft, ob etwas vererbt wird. Wird nichts vererbt, wird der **Standardwert** des **Browserstylesheets** verwendet.
2. Findet der Webbrowser genau **eine Deklaration** einer CSS-Eigenschaft für ein bestimmtes Element, **verwendet** er **diese zum Formatieren des HTML-Elements.**
3. Findet der Webbrowser hingegen **mehrere gleiche** und **konkurrierende** Deklarationen einer CSS-Eigenschaft für ein bestimmtes HTML-Element, werden diese nach ihrer Wichtigkeit, Spezifität und Reihenfolge sortiert und entsprechend dem Ergebnis verwendet. An sich ist dieses Prinzip der drei Quellen hier relativ einfach gehalten, und wenn der Webbrowser keine oder nur eine CSS-Eigenschaft für ein HTML-Element findet, ist die Sache relativ eindeutig. Schwieriger wird es, wenn mehrere konkurrierende Deklarationen zusammentreffen.

# CSS3 | Kaskade (Cascading)

**Die Priorität einer CSS-Eigenschaft mit »!important« erhöhen**

Wenn Sie eine CSS-Regel oder CSS-Eigenschaft mehrmals in derselben Datei deklarieren, erhält gewöhnlich die zuletzt deklarierte Eigenschaft den Zuschlag. Ein einfaches Beispiel hierzu:

Hier wurde **article p** zunächst mit einem 5 Pixel starken grauen und durchgehenden Rahmen formatiert.
Etwas später findet sich erneut eine CSS-Regel dafür, nur wird jetzt **article p** mit einem 10 Pixel starken grünen und gepunkteten Rahmen formatiert.

so funktioniert CSS !important

```
article p{
    margin: 3rem auto;
    padding: 2rem;
    background: beige;
    width: 30%;
    height: auto;
}

article p{
    border: 5px solid gray;
}

article p{
    border: 10px dotted green;
}
```

**Die zuletzt notierte Deklaration erhält den Zuschlag.**

# CSS3 | Kaskade (Cascading)

**Die Priorität einer CSS-Eigenschaft mit !important erhöhen**

Mit dem CSS-Schlüsselwort **!important** hingegen können Sie die Angabe der Priorität erhöhen, sodass diese Eigenschaft von den folgenden Angaben nicht überschrieben werden kann. Wenn Sie dieses Schlüsselwort auf das eben gezeigte Beispiel anwenden: erhält nicht mehr die zuletzt gemachte Deklaration den Vorzug.

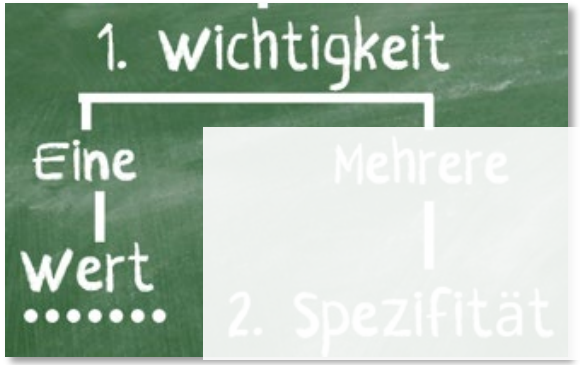
```
article p{
    margin: 3rem auto;
    padding: 2rem;
    background: beige;
    width: 30%;
    height: auto;
}

article p{
    border: 5px solid gray !important;
}

article p{
    border: 10px dotted green;
}
```

so funktioniert CSS !important

In diesem Beispiel wird durch das Schlüsselwort **!important** ein grauer durchgehender Rahmen mit 5 Pixel Stärke um das Element mit der **article p Element** gezeichnet.



# VIELEN DANK!